

UNIT II



Socket Options

There are various ways to get and set the options that affect a socket:

- The getsockopt and setsockopt functions
- The fcntl function

getsockopt and setsockopt Functions

- These two functions apply only to sockets.

```
#include <sys/socket.h>

int getsockopt(int sockfd, int level, int optname, void *optval, socklen_t *optlen);

int setsockopt(int sockfd, int level, int optname, const void *optval,
               socklen_t optlen);
```

Both return: 0 if OK, -1 on error

Parameters – a better understanding

<i>level</i>	<i>optname</i>	get	set	Description	Flag	Datatype
SOL_SOCKET	SO_BROADCAST	•	•	Permit sending of broadcast datagrams	•	int
	SO_DEBUG	•	•	Enable debug tracing	•	int
	SO_DONTROUTE	•	•	Bypass routing table lookup	•	int
	SO_ERROR	•		Get pending error and clear		int
	SO_KEEPALIVE	•	•	Periodically test if connection still alive	•	int
	SO_LINGER	•	•	Linger on close if data to send		linger{}
	SO_OOBINLINE	•	•	Leave received out-of-band data inline	•	int
	SO_RCVBUF	•	•	Receive buffer size		int
	SO_SNDBUF	•	•	Send buffer size		int
	SO_RCVLOWAT	•	•	Receive buffer low-water mark		int
	SO_SNDLOWAT	•	•	Send buffer low-water mark		int
	SO_RCVTIMEO	•	•	Receive timeout		timeval{}
	SO_SNDTIMEO	•	•	Send timeout		timeval{}
	SO_REUSEADDR	•	•	Allow local address reuse	•	int
	SO_REUSEPORT	•	•	Allow local port reuse	•	int
	SO_TYPE	•		Get socket type		int
	SO_USELOOPBACK	•	•	Routing socket gets copy of what it sends	•	int
IPPROTO_IP	IP_HDRINCL	•	•	IP header included with data	•	int
	IP_OPTIONS	•	•	IP header options		(see text)
	IP_RECVDSTADDR	•	•	Return destination IP address	•	int
	IP_RECVIF	•	•	Return received interface index	•	int
	IP_TOS	•	•	Type-of-service and precedence		int
	IP_TTL	•	•	TTL		int
	IP_MULTICAST_IF	•	•	Specify outgoing interface		in_addr{}
	IP_MULTICAST_TTL	•	•	Specify outgoing TTL		u_char
	IP_MULTICAST_LOOP	•	•	Specify loopback		u_char
	IP_{ADD,DROP}_MEMBERSHIP		•	Join or leave multicast group		ip_mreq{}
	IP_{BLOCK,UNBLOCK}_SOURCE		•	Block or unblock multicast source		ip_mreq_source{}
	IP_{ADD,DROP}_SOURCE_MEMBERSHIP		•	Join or leave source-specific multicast		ip_mreq_source{}



```
int getsockopt(int sockfd, int level, int optname, void *optval, socklen_t *optlen);
```

Parameter

Description

sockfd

Socket descriptor whose option is to be **read**.

level

Protocol level where the option is defined (e.g., SOL_SOCKET, IPPROTO_TCP).

optname

Name of the socket option to retrieve (e.g., SO_REUSEADDR, SO_KEEPALIVE).

optval

Pointer to the **buffer where the option value is stored**.

optlen

Pointer to the size of optval; updated with the actual size returned.



Level Protocol level where the option is defined

Level	Description	Examples of Options (<i>optname</i>)
SOL_SOCKET	Socket-level options that apply to all socket types (TCP and UDP).	SO_REUSEADDR, SO_KEEPALIVE, SO_BROADCAST, SO_LINGER, SO_RCVBUF, SO_SNDBUF
IPPROTO_IP	IPv4 protocol-level options.	IP_TTL, IP_TOS, IP_MULTICAST_TTL
IPPROTO_IPV6	IPv6 protocol-level options.	IPV6_V6ONLY, IPV6_UNICAST_HOPS, IPV6_MULTICAST_HOPS
IPPROTO_TCP	TCP-specific options.	TCP_NODELAY, TCP_MAXSEG, TCP_KEEPIIDLE
IPPROTO_UDP	UDP-specific options (less commonly used).	UDP-related socket options (implementation dependent)

How level and optname Work Together

The optname parameter specifies which socket option to be set or retrieve.

Its meaning depends on the level parameter.

For example, if level is SOL_SOCKET, then optname must be a socket-level option such as SO_REUSEADDR or SO_KEEPALIVE.

Level	Valid optname Examples
SOL_SOCKET	SO_REUSEADDR, SO_KEEPALIVE, SO_BROADCAST, SO_LINGER
IPPROTO_IP	IP_TTL, IP_TOS, IP_MULTICAST_TTL
IPPROTO_IPV6	IPV6_V6ONLY, IPV6_UNICAST_HOPS
IPPROTO_TCP	TCP_NODELAY, TCP_MAXSEG



```
int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen);
```

Parameter	Description
sockfd	Socket descriptor whose option is to be modified .
level	Protocol level where the option is defined (e.g., SOL_SOCKET, IPPROTO_TCP).
optname	Name of the socket option to set (e.g., SO_REUSEADDR, SO_KEEPALIVE).
optval	Pointer to the new value of the socket option .
optlen	Size (in bytes) of the option value pointed to by optval.

Return Value

- **0** → Success
- **-1** → Error

Socket States

- Some socket options must be set **before the** TCP connection is established.
- In TCP, the connected socket inherits certain options from the listening socket.
- **Since `accept()` returns only after the TCP three-way handshake completes, these options must be configured on the listening socket beforehand.**
- Common inherited options include: `SO_KEEPALIVE`, `SO_REUSEADDR`, `SO_LINGER`, `SO_RCVBUF`, `SO_SNDBUF`, `TCP_NODELAY`, `TCP_MAXSEG`, etc.
- Importance:
 - Setting these options on the listening socket ensures that every newly accepted client connection automatically uses the same settings.

Linger means **to remain, stay, or wait for a short time before leaving or ending.**
Stay longer than expected or delay leaving.

Generic Socket Options

- Generic socket options are common **options that control the behavior of sockets** and can be applied using the `setsockopt()` and `getsockopt()` system calls.
- These options improve performance, reliability, buffering, debugging, timeout handling, and connection management.
- Socket options are organized into different protocol levels, such as `SOL_SOCKET` (socket-level), `IPPROTO_IP` (IPv4), `IPPROTO_IPV6` (IPv6), and `IPPROTO_TCP` (TCP).
- Some options must be set before calling `bind()`, `listen()`, or `connect()`, while others can be modified after the connection is established.
- For TCP servers, many socket options set on the listening socket are automatically inherited by the connected socket returned by `accept()`.



Socket Option	Description
SO_BROADCAST	Enables a socket to send broadcast datagrams on networks that support broadcasting. Commonly used with UDP for local network broadcasts. TCP, SCTP do not support broadcasting
SO_DEBUG	Requests the kernel to record debugging information for the socket. Mainly used for troubleshooting and protocol debugging. UDP does not support
SO_KEEPALIVE	Enables the TCP keepalive mechanism to periodically check whether the connected peer is still alive. If no response is received from a connected socket for 2 hrs, a probe is sent. No response – 8 probes sent 75 seconds apart. If the peer does not respond to the keepalive probes, TCP closes the connection and returns an error to the application.
SO_DONTROUTE	Sends packets directly to the local network without using routing tables. Useful for testing or communication with directly connected hosts.
SO_ERROR	Retrieves and clears any pending error associated with the socket. Helps applications determine the cause of socket failures.
SO_LINGER	Controls the behavior of close() when unsent data remains in the socket buffer. It can force a graceful close or an immediate connection reset.



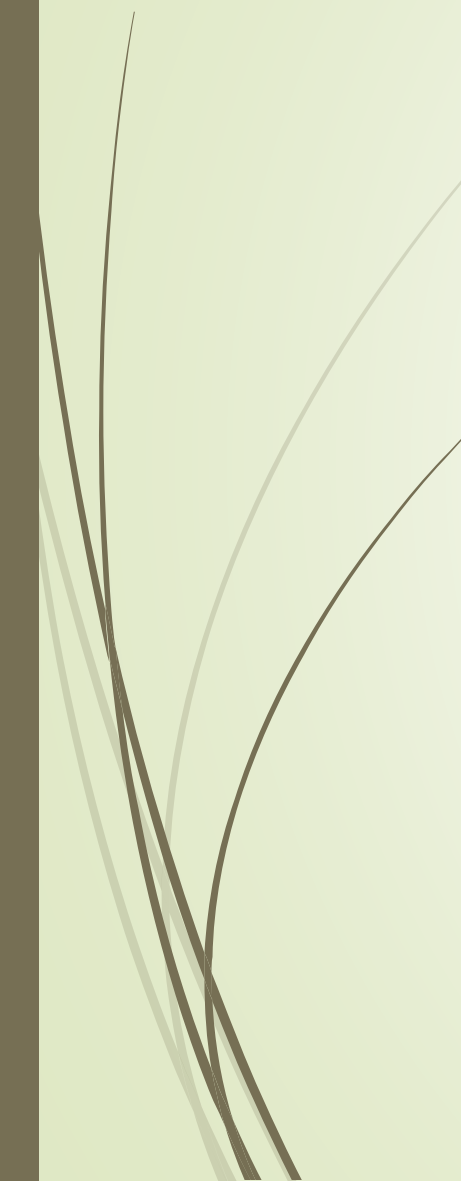
Socket Option	Description
SO_OOBINLINE	Keeps TCP out-of-band (urgent) data in the normal data stream. The application can read urgent data using ordinary read operations.
SO_RCVBUF / SO_SNDBUF	Set or get the receive and send buffer sizes of a socket. Larger buffers can improve throughput for high-speed communication.
SO_RCVLOWAT / SO_SNDLOWAT	Specify the minimum number of bytes required for receive or send operations to proceed. They define low-water marks for socket I/O.
SO_RCVTIMEO / SO_SNDTIMEO	Set timeouts for blocking receive and send operations. If the timeout expires, the operation returns an error instead of waiting indefinitely.
SO_REUSEADDR / SO_REUSEPORT	Allow reuse of local addresses and, in some systems, ports. Useful for restarting servers quickly and for load sharing among processes.
SO_TYPE	Returns the type of socket, such as SOCK_STREAM or SOCK_DGRAM. Applications use it to verify socket characteristics.
SO_USELOOPBACK	Causes transmitted packets to use the loopback interface when appropriate. Primarily intended for local host communication and testing.

IPv4 Socket Options

Socket Option	2-Line Summary
IP_HDRINCL	Allows the application to provide its own IPv4 header for outgoing packets. Used mainly with raw sockets for custom packet generation.
IP_OPTIONS	Sets or retrieves IPv4 header options such as source routing, record route, or timestamp. It allows special processing of IP packets.
IP_RECVSTADDR	Causes the destination IP address of a received datagram to be returned with the message. Useful when a host has multiple IP addresses.
IP_RECVIF	Returns the interface on which a datagram was received. Helps applications determine the incoming network interface.
IP_TOS	Sets the IPv4 Type of Service (ToS) or Differentiated Services field. It can request priority, low delay, or higher throughput treatment for packets.
IP_TTL	Sets the Time-To-Live value for outgoing IPv4 packets. The TTL limits how many routers a packet can pass through before being discarded.

IPv6 Socket Options

Socket Option	2-Line Summary
IPV6_CHECKSUM	Sets or retrieves the checksum offset for raw IPv6 sockets. Used by applications that generate their own transport headers.
IPV6_DONTFRAG	Prevents fragmentation of outgoing IPv6 packets. If a packet exceeds the MTU, it is dropped and an ICMPv6 "Packet Too Big" message is generated.
IPV6_NEXTHOP	Specifies the next-hop IPv6 address for outgoing packets. It allows applications to override the default routing decision.
IPV6_PATHMTU	Retrieves the current Path MTU for a connected IPv6 socket. It helps applications avoid packet fragmentation.
IPV6_RECVDSTOPTS	Enables receiving IPv6 Destination Options with incoming packets. Applications can process optional extension header information.
IPV6_RECVHOPLIMIT	Returns the Hop Limit value of received IPv6 packets. Similar to receiving the TTL value in IPv4.
IPV6_RECVHOPOPTS	Delivers Hop-by-Hop extension headers to the application. These headers contain options processed by every router.



Socket Option	Description
IPV6_RECVPATHMTU	Notifies the application whenever the Path MTU changes. Useful for dynamic Path MTU Discovery.
IPV6_RECVPKTINFO	Returns packet information such as the destination IPv6 address and receiving interface. Useful on multihomed hosts.
IPV6_RECVRTHDR	Delivers the IPv6 Routing Header to the application. It is used when source routing information is present.
IPV6_RECVTCLASS	Returns the IPv6 Traffic Class field with received packets. It allows applications to examine QoS information.
IPV6_UNICAST_HOPS	Sets or retrieves the Hop Limit for outgoing unicast IPv6 packets. It performs the same role as the IPv4 TTL field.
IPV6_USE_MIN_MTU	Controls whether outgoing packets use the minimum IPv6 MTU (1280 bytes). Useful on networks with small MTUs.
IPV6_V6ONLY	Restricts an IPv6 socket to communicate only with IPv6 addresses. It prevents the socket from accepting IPv4-mapped IPv6 addresses.
IPV6_XXX	Represents other implementation-specific IPv6 socket options supported by the operating system. These options provide additional IPv6 features and controls.



TCP Socket Options

TCP_MAXSEG Socket Option

- TCP_MAXSEG allows the application to retrieve or set the Maximum Segment Size (MSS) used by TCP for sending data.
- The returned value is generally based on the MSS advertised by the peer, but TCP may use a smaller value due to TCP options or network conditions.
- The effective MSS can change during a connection when Path MTU Discovery detects a different path MTU.
- On systems that allow setting it, the application can generally reduce the MSS but cannot increase it beyond the peer's advertised limit.



TCP_NODELAY Socket Option

- Nagle algorithm is enabled by default and reduces the number of small TCP packets by waiting for the acknowledgment of previously sent data before sending another small packet.
- TCP_NODELAY disables the Nagle algorithm, allowing small packets to be sent immediately; this can reduce delay but may increase network traffic.
- Nagle works well on WANs by reducing small packets, but it can cause noticeable delays in interactive applications such as Telnet and Rlogin.
- Nagle can interact with TCP delayed ACK, causing additional delays when the server has no data to send back.
- If an application sends one logical request in multiple small pieces, `writv()` or a single `write()` is preferred over disabling Nagle with TCP_NODELAY.
- TCP_NODELAY is useful when low latency is more important than reducing the number of TCP packets.